

FastAPI 基本概念

學習目標

完成本單元後，你將能夠：

1. 說明 FastAPI 是什麼、適合用在哪裡
2. 從零建立一個 FastAPI 專案
3. 撰寫第一支 API 程式碼
4. 在本機啟動伺服器並驗證結果

一、什麼是 FastAPI？

FastAPI 是一個用 **Python** 寫後端 API 的現代化框架，特色包含：

特點	說明
快速	效能接近 Node.js、Go，適合高併發
易寫	語法簡潔，型別提示 (Type Hints) 就能自動驗證資料
自動文件	啟動後會自動產生 Swagger UI (/docs)
非同步	支援 <code>async / await</code> ，適合 I/O 密集場景

常見使用情境

- RESTful API (前後端分離)
- 微服務後端
- 搭配前端模板 (Jinja2) 做簡單網站
- AI / 資料服務的 API 層

核心組成

FastAPI (框架)

└─ Uvicorn (ASGI 伺服器，負責真正把程式跑起來)

- **FastAPI** : 定義路由、處理請求、回傳回應
- **Uvicorn** : 把你的 FastAPI 應用變成可對外服務的 HTTP 伺服器

二、建立專案

1. 確認 Python 版本

建議使用 **Python 3.10+** :

```
python3 --version
```

2. 建立專案資料夾

```
mkdir fastapi-hello  
cd fastapi-hello
```

3. 建立虛擬環境 (強烈建議)

虛擬環境可避免套件污染系統 Python :

```
# macOS / Linux  
python3 -m venv .venv  
source .venv/bin/activate
```

```
# Windows  
python -m venv .venv  
.venv\Scripts\activate
```

啟用成功後，終端機前面通常會出現 **(.venv)** 。

4. 安裝必要套件

建立 requirements.txt :

```
fastapi==0.115.6
uvicorn[standard]==0.34.0
```

安裝：

```
pip install -r requirements.txt
```

或直接安裝：

```
pip install "fastapi[standard]"
```

5. 建議的專案結構 (最小版)

```
fastapi-hello/
├── .venv/           # 虛擬環境 ( 不要提交到 Git )
├── main.py          # 主程式
└── requirements.txt # 依賴清單
```

三、基本程式碼

在專案根目錄建立 main.py :

```
from fastapi import FastAPI

# 建立 FastAPI 應用實例
app = FastAPI(title="我的第一個 FastAPI")


@app.get("/")
async def home():
    """首頁：回傳簡單的 JSON"""
    return {"message": "Hello, FastAPI!"}


@app.get("/hello/{name}")
async def hello(name: str):
    """路徑參數：把 URL 中的 name 帶進來"""
    return {"message": f"你好 · {name}!"}


@app.get("/items")
async def list_items(limit: int = 10):
    """查詢參數：例如 /items?limit=3"""
    items = ["拿鐵", "美式", "卡布奇諾", "摩卡", "可頌"]
    return {"items": items[:limit]}
```

程式碼重點說明

概念	說明
<code>app = FastAPI(...)</code>	建立應用本體，之後所有路由都掛在它上面
<code>@app.get("/")</code>	裝飾器：宣告「這個函式處理 GET / 」
<code>async def</code>	非同步函式；簡單範例也可用一般 <code>def</code>
路徑參數 <code>{name}</code>	從 URL 路徑取值，例如 <code>/hello/小明</code>
查詢參數 <code>limit</code>	從 <code>?limit=3</code> 取值，可設預設值

HTTP 方法對照 (常用)

方法	用途	FastAPI 寫法
GET	查詢資料	<code>@app.get(...)</code>
POST	新增資料	<code>@app.post(...)</code>
PUT	更新資料	<code>@app.put(...)</code>
DELETE	刪除資料	<code>@app.delete(...)</code>

再看一個 POST 範例 (可選)

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

@app.post("/users")
async def create_user(user: User):
    return {"ok": True, "user": user}
```

請求時送出 JSON :

```
{
  "name": "小明",
  "age": 20
}
```

FastAPI 會自動驗證欄位型別；缺欄位或型別錯誤會回傳 422。

四、啟動專案

方式一：uvicorn 指令（課堂常用）

在專案根目錄（有 `main.py` 的地方）執行：

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

參數說明：

參數	意義
<code>main:app</code>	<code>main.py</code> 裡的 <code>app</code> 變數
<code>--reload</code>	改程式後自動重載（開發用）
<code>--host 0.0.0.0</code>	允許其他裝置連線（本機可省略）
<code>--port 8000</code>	監聽埠號

看到類似訊息即表示成功：

```
INFO:      Uvicorn running on http://127.0.0.1:8000
INFO:      Application startup complete.
```

方式二：在程式中啟動（較少用）

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="127.0.0.1", port=8000, reload=True)
```

然後：

```
python main.py
```

五、如何驗證有沒有跑起來？

啟動後用瀏覽器或終端機測試：

網址	預期結果
http://127.0.0.1:8000/	<code>{"message": "Hello, FastAPI!"}</code>
http://127.0.0.1:8000/hello/元智	<code>{"message": "你好，元智！"}</code>
http://127.0.0.1:8000/items?limit=2	回傳前 2 個品項
http://127.0.0.1:8000/docs	自動產生的 API 文件 (Swagger UI)
http://127.0.0.1:8000/redoc	另一種文件介面 (ReDoc)

用 curl 測試：

```
curl http://127.0.0.1:8000/
curl http://127.0.0.1:8000/hello/小美
curl "http://127.0.0.1:8000/items?limit=2"
```

六、課堂練習

練習 1：改訊息

把首頁的 Hello, FastAPI! 改成你的名字歡迎詞，重新整理瀏覽器確認 (有 `--reload` 時不用重啟) 。

練習 2：新增路由

新增 GET /about ，回傳：

```
{
  "course": "FastAPI 入門",
  "school": "元智大學"
}
```

練習 3：路徑參數

新增 GET /price/{item}，輸入品名後回傳假價格，例如：

- /price/拿鐵 → {"item":"拿鐵","price":120}

七、常見問題

問題	可能原因	解法
command not found: uvicorn	虛擬環境未啟用，或未安裝	source .venv/bin/activate 後再 pip install uvicorn
Address already in use	8000 埠被占用	改用 --port 8001
改程式沒反應	沒加 --reload	加上 --reload，或手動重啟
ImportError / ModuleNotFoundError	套件沒裝、 或開錯資料夾	確認在專案目錄、虛擬環境已啟用
/docs 打不開	伺服器沒起來	先確認終端機有 Application startup complete

八、本單元重點整理

1. FastAPI 用來寫 API；真正對外服務靠 **Uvicorn**
2. 建立專案流程：資料夾 → 虛擬環境 → 安裝套件 → 寫 main.py
3. 用 @app.get / @app.post 宣告路由
4. 啟動指令：uvicorn main:app --reload
5. 用 /docs 快速測試與查看 API 文件

下一步

完成本單元後，請繼續：

1. [02-路由進階](#) — Path / Query / Body、狀態碼、APIRouter
2. [04-Pydantic-Model](#) — 資料驗證與 Schema (可與 02 穿插)
3. [03-Jinja2模板](#) — 做出咖啡廳網頁
4. [05-依賴注入](#) — 共用邏輯與權限

最終可對照根目錄「Brew & Bean」專案，並參考 `md/指令講義.md` 部署到 Cloud Run。